

Fuzzing Libsixel

A report for CS-412

Maria Bennani
EPFL

Sean Perazzolo
EPFL

Valuthy Karunakaran
EPFL

Introduction

This report details the design, executions and analysis of a coverage guided fuzzing campaign on the open-source library `libsixel` using the `AFL++` fuzzer. The target repository was pinned to version 1.8.7, to ensure reproducibility.

`libsixel` is an encoder/decoder library for `SIXEL`, an image format for printer and terminal imaging.

The project is comprised of two parts: white-box fuzzing (source-instrumented) and black-box (binary-only, using `QEMU`).

1 Harness Design

1.1 Selected Entrypoint And Considered Alternatives

For this campaign, we selected the **encoding path** of the library, specifically targeting the `sixel_encoder_encode` function.

Even though the guidelines suggested fuzzing the decoding path, recently identified vulnerabilities were due to bugs in the encoding path. This made the encoder a highly attractive target.

We initially considered fuzzing the high-level `img2sixel` function. However, we rejected this approach because high-level wrappers introduce unnecessary overhead (e.g. command-line argument parsing, I/O operations, ...). By writing a custom harness directly around `sixel_encoder_encode`, we skip all non-encoding related functionality.

We also considered using a PNG corpus, but due to strict checks by the parsing library (`libpng`), our fuzzer was not able to reach the encoding logic. Inspired by the recent CVE, we switched to a corpus of GIF files, which are not subject

to such strict checks and are still able to trigger the encoding logic.

1.2 Data Flow and Guards

Each `AFL++` test case is split into two parts: a 8-byte control header and the raw image payload.

The first 8-bytes are parsed by the harness and mapped to configuration options. They are passed to the library via `sixel_encoder_setopt` to configure encoder options such as color mode, quality, diffusion, resize/crop behavior, and animation-related flags.

The remaining bytes represents the image and are written to a temporary file. The latter is passed to `sixel_encoder_encode`, which expects a file path as input. Note that the harness receives each `AFL++` test case through `stdin`, rather than through the usual `@@ filename` argument, avoiding I/O overhead.

The main data flow is therefore:

```
1  AFL++ testcase -> stdin -> [8-byte option header] + [GIF
    payload] -> temporary GIF file -> sixel_encoder_encode
```

This lets `AFL++` mutate both the image contents and the encoder configuration. The option bytes are mapped to fixed valid strings, to ensure the fuzzer explores many combinations of `libsixel` behavior.

The harness contains several guards to allow the fuzzer to efficiently explore the library:

1. A guard ensures that the test case is at least 8 bytes long to contain the control header. This is needed because the harness expects the first 8 bytes to be valid options, and shorter inputs would be rejected before reaching the library code.
2. A guard ensures that the test case also contains a complete 6-byte GIF header after the control bytes. This is needed because the harness reconstructs an image

file before calling the library API. Inputs shorter than this would only exercise trivial file rejection and would not reach the interesting decoder or encoder code. *Note that this guard can be removed to allow broader exploration of the file parsing logic with* `make FUZZ_BROAD_LOADER=1`.

3. We then have various other guards to ensure the harness passes valid options / files to the library, such as `if (fd < 0) continue;` to ensure the file has been successfully written or `if (status != SIXEL_OK) continue;` which ensures the option values are valid and accepted by the library.

We also guard all harness option selection by mapping arbitrary AFL++ bytes into finite option sets using masks or modulo operations. This keeps crashes attributable to the library code and not to the harness (e.g. by preventing invalid option values from being passed to the library).

Finally the resize and crop options are chosen from small fixed dictionaries. This bounds generated image dimensions and crop regions, preventing AFL++ from spending time on huge allocations, very slow transformations, or timeout-heavy inputs while still exercising scaling, cropping, and resampling code.

2 Instrumentation and Sanitizers

2.1 Target Version

We built **libsixel v1.8.7**, the latest stable release of the library. At the time of writing, two releases candidates were published (r1, r2), they address security vulnerabilities in the encoding path: These CVEs acts as a benchmark for our campaign as we should be able to find them with our setup, and they provide a good case study, as well as motivate our choice of fuzzing the encoder instead of the decoder.

2.2 Coverage Instrumentation

We instrumented the library with AFL++'s `afl-clang-lto / afl-clang-lto++`, which inserts compile-time edge coverage. This gives AFL++ precise feedback for input selection.

```
1 CC=afl-clang-lto CXX=afl-clang-lto++
```

2.3 AddressSanitizer

We also built the library with AddressSanitizer (ASan) to detect memory safety bugs. This allows us to catch memory corruption issues that might not immediately cause a crash, improving our chances of finding security vulnerabilities.

If omitted, AFL++ would still find hard crashes, but many invalid reads/writes would remain silent or become harder-to-reproduce later crashes.

```
1 ENV AFL_USE_ASAN=1
```

Because ASan reserves a large virtual address space, we run AFL++ with `-m none`; otherwise AFL++'s default memory limit can kill valid ASan-instrumented executions.

At runtime, we used the following `ASAN_OPTIONS`

```
1 detect_leaks=0
2 abort_on_error=1
3 symbolize=0
```

Leak detection was disabled because the persistent harness intentionally runs many iterations in one process, and leak reports at process exit are less useful for this campaign than immediate memory-safety crashes. `abort_on_error=1` ensures sanitizer findings terminate the process in a way AFL++ records as a crash. `symbolize=0` avoids the overhead of online symbolization during fuzzing; symbolization can be done later during triage.

2.4 Build Configuration

The flags `-g -O2` were used to include debug symbols for better crash triage and to optimize the code for more realistic performance.

`--prefix=/usr/local` ensures the harness links against the freshly built instrumented library. `--with-gd`, `--with-jpeg`, and `--with-png` enable the image backends relevant to realistic encoder inputs; `--with-gd` is especially important for GIF handling. `--with-libcurl=no`, `--with-gdk-pixbuf2=no`, and `--disable-python` remove network, GUI, and language-binding components that our harness does not exercise.

2.5 Linking

When linking the final harness, we used `-Wl,--whole-archive -lsixel -Wl,--no-whole-archive`. This forces the linker to include the full static `libsixel` archive in the final binary instead of only the object files directly referenced by the harness. This is useful for instrumentation accounting and comparison because the final binary contains the library code rather than only a minimal linker-selected subset. Without `--whole-archive`, the binary would be smaller, but some instrumented library edges would be absent from the final executable.

2.6 Patching

The library was configured with `--disable-shared --enable-static`. This makes the harness link against a static `libsixel` archive instead of a shared library. Static linking makes the fuzzing binary self-contained and ensures that the instrumented library code is the code executed by AFL++. If we used an uninstrumented shared system library by mistake, AFL++ would mainly see coverage from the harness rather than from the target library.

We did not apply a source patch to `libsixel`. In particular, there was no checksum-removal patch comparable to the `libpng` CRC patch discussed in the handout. Our main format was GIF, whose magic-byte check is handled at the harness level by preserving or filtering for GIF87a/GIF89a.

The optional `TRUEVISION_PATCH=1` setting in our setup is not a patch to the target library; it is a harness-side filter that skips very small image payloads to avoid shallow file-detection behavior and focus executions on deeper parsing paths.

3 Seed Corpus and Dictionary

We selected a raw seed corpus of 10 small GIF files that cover a variety of features of the GIF format. The corpus includes both GIF87a and GIF89a files, 1x1 edge-case images, transparent GIFs, interlaced and non-interlaced variants of the same image, small 32x32 thumbnails, and larger 100x100 images. This gave AFL++ structurally valid starting points while keeping dry-run and mutation cost low.

Two seeds were selected in particular because they are proof-of-concept inputs for the vulnerability classes we aimed to rediscover:

- `poc_load_gif.gif`: a 32x32 two-frame animated GIF. This exercises the multi-frame GIF loading path and is relevant to the `load_gif()` use-after-free class of bugs.
- `pocl_gif_oob.gif`: a minimal 1x1 GIF. This seed stresses boundary conditions in the image loader and conversion path, especially tiny dimensions and short image data.

Before fuzzing, each raw GIF is wrapped with the 8-byte harness control header described earlier, yielding inputs of the form `control header || GIF payload`.

To keep the baseline campaign fast, we excluded expensive seeds from the default corpus, they can optionally be included by using the appropriate flag when running the campaign.

3.1 Mutation Strategy Analysis

The AFL++ strategy-yield table shows that dictionary-based mutations contributed 136 new paths. The dictionary row reports 136/458k, 0/459k, 0/0, 0/0, meaning that the first dictionary mutation mode found 136 interesting inputs over about 458k executions. The havoc/splice row reports 973/1.88M, 0/0, so havoc contributed 973 new paths over about 1.88M executions, while splicing did not contribute in this run, AFL++ did not spent executions in the splice stage.

The numbers are reported in Table 3

4 Campaign Analysis

The main greybox campaign **ran for an hour**. The AFL++ status screen is shown in Figure 1, and the `afl-plot` graphs are shown in Figure 2, Figure 3, Figure 4, and Figure 5.

At the end of the run, AFL++ reported a **corpus count of 1515 inputs**, with 122 favored items and 242 inputs that discovered new edges.

The campaign completed two full queue cycles (not shown in the status screen). It also found **51 unique crashes** and 58 saved hangs.

AFL++ reported **99.96% stability**, which means that repeated executions of the same inputs almost always produced the same coverage.

The bitmap density was 10.34% for the whole corpus, with 5.44 bits per tuple. This indicates that the campaign reached a meaningful part of the instrumented program, while still remaining far from bitmap saturation.

The edge curve rises quickly at the beginning of the run and then continues as smaller step increases. This is the expected shape for a useful fuzzing campaign: AFL++ first discovers shallow parsing paths from the initial GIF seeds, then later reaches deeper behavior through mutation and queue cycling.

The status screen reports that a new path had been found 58 seconds before the screenshot, so the campaign was still productive when it was stopped. For this reason, **we do not consider the campaign saturated**. The run is long enough to validate the harness and demonstrate that AFL++ reaches meaningful decoder behavior, especially since it found 51 unique crashes. However, because new paths were still being found near the end of the hour long run, a longer campaign, would likely discover additional coverage and possibly more crashes.

5 Crash Triage

Crash selected: AFL++ crash id:000047 from outputs/20260514-142308/default/crashes. **fix path to put final path of the results**

One of the two crashes we found was a **heap-based buffer overflow**, specifically it is an **out-of-bounds write** in libsixel’s GIF decoder. The input reaches `load_gif()`, which calls `gif_init_frame()`. AddressSanitizer reports an out-of-bounds write at `fromgif.c:241`:

```
frame->palette[pg->transparent * 3 + 0] =
bgcolor[0];
```

The root cause is that the GIF transparency index from the Graphic Control Extension is used as an index into `frame->palette` without checking that it is smaller than the number of palette entries. In the crashing input, the transparent index is `0xdf`, while the GIF is a tiny `1x1` image with a much smaller palette. This causes writes past the heap allocation for `frame->palette`.

Under ASan the program aborts with SIGABRT, without ASan the bug may corrupt adjacent heap memory.

This Bug does not have a assigned CVE but has been already reported and fixed in the release candidate: <https://github.com/saitoha/libsixel/issues/220>.

The triage included reproducing the bug with our harness, then with `img2sixel` and finally with a minimized version.

All commands necessary for the full triage, as well as the stacktrace are in the Appendix Section 9.4

6 Attack Surface Analysis

Name two real-world applications that use your target library and describe a concrete attack scenario. Then identify at least two code paths in those applications that your harness does not exercise, and explain why these gaps matter for security.

7 Binary-Only Fuzzing with QEMU Mode

To evaluate the library in a black-box scenario, we ran a campaign against an uninstrumented binary using AFL++ QEMU mode (`-Q`). We built vanilla versions of the harness and `libsixel` (v1.8.7) using standard `gcc / g++`, confirming via `nm` and through the library build configurations that no sanitizer symbols or instrumentation points were present.

The full performance comparison after 300 seconds is reported in Table 4.

The discrepancies between these metrics come from the differences in how each mode collects coverage and executes the target:

1. **Execution Speed:** The instrumented campaign is $\sim 30x$ faster. This is primarily due to the **persistent mode** (`__AFL_LOOP`), which allows the fuzzer to reuse the same process for multiple test cases. In contrast, the QEMU campaign lacks persistent mode and has the significant overhead of Just-In-Time (JIT) binary translation for every instruction.
2. **Edges Discovered:** QEMU mode discovered $\sim 35\%$ more edges despite having $30x$ fewer executions. This is because compile-time instrumentation only sees branches in the source code it compiled. QEMU mode instruments the entire process address space during emulation, capturing paths within shared system libraries (e.g., `libpng`, `libjpeg`, `libc`) that are black-boxes to the instrumented version.
3. **Corpus Count:** The native fuzzer’s higher throughput allowed it to explore a larger mutation space, leading to a larger corpus within the same timeframe.

Despite the performance penalty, the QEMU campaign, when combined with QASan (QEMU-AddressSanitizer), successfully identified memory safety issues. By disabling the `TRUEVISION` patch (which previously filtered out small inputs), we reproduced a heap-buffer underflow in the file format detection logic. While a vanilla binary might “silently” corrupt memory without crashing, QASan intercepts memory-related library calls (like `memcpy`) and validates their arguments against a shadow memory map, promoting these “soft” corruptions to detectable crashes.

8 Instrumentation Depth and Performance

Report the number of instrumented edges reported by `afl-fuzz` at startup for (a) the library alone and (b) the final harness binary. Explain why these numbers differ. Then compare your campaign’s map density to the total instrumented edges and explain why not all edges were reached. Additionally, measure `exec speed` under three configurations using the same harness: (1) no sanitizer + fork mode, (2) ASan + fork mode, (3) ASan + persistent mode. Report the three numbers and explain the source of each speedup or slowdown.

9 Appendix

9.1 Fuzzing Speed Measurements

Profile	Loader	Time	Execs	Speed
fast	0	74s	241,802	3235.72 exec/s
extended	0	73s	195,386	2656.29 exec/s
fast	1	74s	124,718	1670.21 exec/s
extended	1	73s	51,441	698.60 exec/s

Table 1: AFL++ speed comparison across seed profile and loader configurations.

Metric	-t 1000	-t 5000
Run time	74s	74s
Execs done	227,021	215,103
Speed	3038.45 exec/s	2878.36 exec/s
Corpus count	717	762
Bitmap coverage	9.34%	9.61%
Edges found	1853	1905
Crashes	26	29
Hangs	8	0

Table 2: Timeout A/B test using SEED_PROFILE=fast
BROAD_LOADER=0.

9.2 Fuzzing Strategy Yields

Strategy	Yield	Impact
Bit flips	29/40.3k, 20/40.3k, 16/40.2k	Small
Byte flips	2/5035, 2/5025, 4/5005	Small
Arithmetics	86/351k, 31/696k, 27/693k	Moderate
Known ints	3/45.1k, 5/189k, 16/279k	Small
Dictionary	136/458k, 0/459k, 0/0, 0/0	Useful
Havoc/splice	973/1.88M, 0/0	Dominant
Py/custom/rq	unused, unused, unused, unused	Not used
Trim/eff	8.57%/481k, 98.83%	Corpus cleanup

Table 3: AFL++ strategy-yield summary. Havoc produced the most interesting inputs, while dictionary mutations also contributed meaningful new paths.

9.3 Campaign analysis

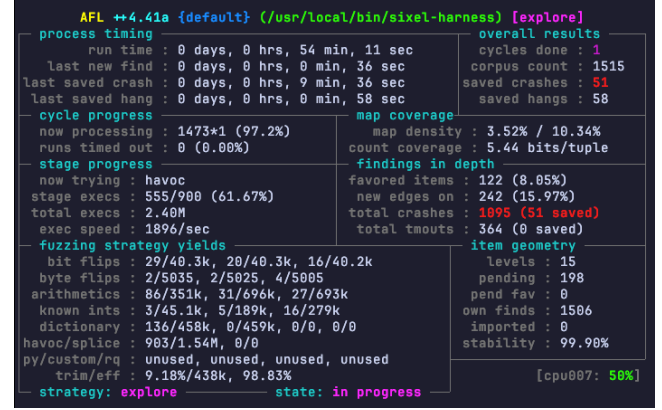


Figure 1: AFL++ status screen during the greybox campaign.

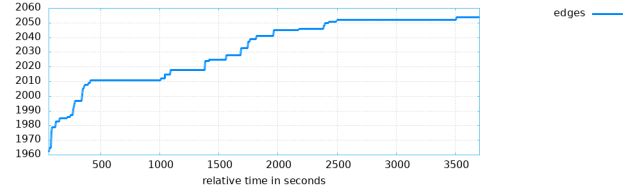


Figure 2: AFL++ edge coverage over time.

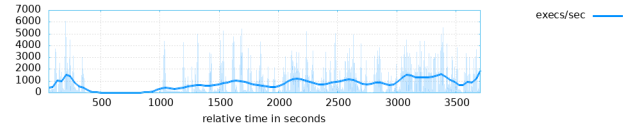


Figure 3: AFL++ execution speed over time.

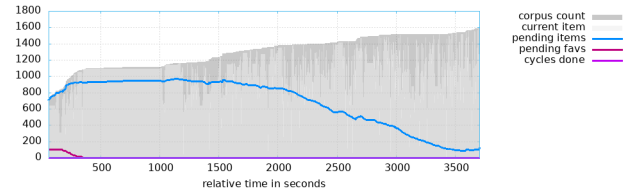


Figure 4: High-frequency AFL++ plot metrics.

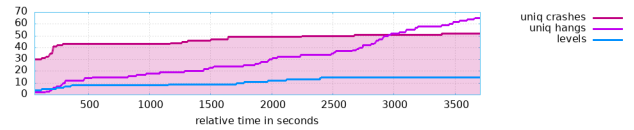


Figure 5: Low-frequency AFL++ plot metrics.

9.4 Crash triage

```
1 ERROR: AddressSanitizer: heap-buffer-overflow
2 WRITE of size 1
3 #0 0x... in gif_init_frame /src/libsixel/src/
  fromgif.c:241:61
4 #1 0x... in load_gif /src/libsixel/src/fromgif.c:675:22
5 #2 0x... in load_with_built_in /src/libsixel/src/
  loader.c:948:18
6 #3 0x... in sixel_helper_load_image_file /src/libsixel/src/
  loader.c:1462:18
7 #4 0x... in sixel_encoder_encode /src/libsixel/src/
  encoder.c:1816:14
8 #5 0x... in main /src/harness.c:215:9
9
10 SUMMARY: AddressSanitizer: heap-buffer-overflow /src/
  libsixel/src/fromgif.c:241:61 in gif_init_frame
```

Listing 1: Backtrace of crash 47.

9.4.1 afl-tmin

```
1 mkdir -p triage
2
3 docker run --rm \
4 -v "$PWD:/host" \
5 -e ASAN_OPTIONS=detect_leaks=0:abort_on_error=1:symbolize=1
  \
6 libsixel-fuzzer \
7 afl-tmin -m none -t 1000 \
8 -i '/host/outputs/20260514-142308/default/crashes/
  id:000047...' \
9 -o /host/triage/id000047.min \
10 -- /usr/local/bin/sixel-harness
```

Listing 2: Command to minimise the seed.

```
1 Read 51 bytes from input.
2 Program exits with a signal, minimizing in crash mode.
3 File size reduced by: 23.53% (to 39 bytes)
4 Characters simplified: 69.23%
5 Number of execs done: 249
6 Output written to: triage/id000047.min
```

Listing 3: Afl-tmin result.

```
1 docker run --rm \
2 -v "$PWD:/host" \
3 -e ASAN_OPTIONS=detect_leaks=0:abort_on_error=1:symbolize=1 \
4 libsixel-fuzzer \
5 /bin/bash -lc '/usr/local/bin/sixel-harness < /host/triage/
  id000047.min'
```

Listing 4: Reproducing the crash with minimised seed.

```
1 docker run --rm \
2 -v "$PWD:/host" \
3 -e ASAN_OPTIONS=detect_leaks=0:abort_on_error=1:symbolize=1
  \
4 libsixel-fuzzer \
5 /bin/bash -lc 'img2sixel -o /dev/null -g -p 256 -q auto -d
  auto -t rgb -f auto -s auto -E auto -B "#FF0000" -l auto /
  host/triage/id000047.min.gif'
```

Listing 5: Running img2sixel with minimised input

9.5 QEMU Performance Comparison

Axis	Instrumented	QEMU mode
Exec speed	2,670.5 exec/s	90.1 exec/s
Edges discovered	2,001	2,696
Corpus count	1,151	565

Table 4: Performance comparison after 300 seconds of wall-clock time.